AI ASSISSTED CODING LAB ASSIGNMENT – 20.3

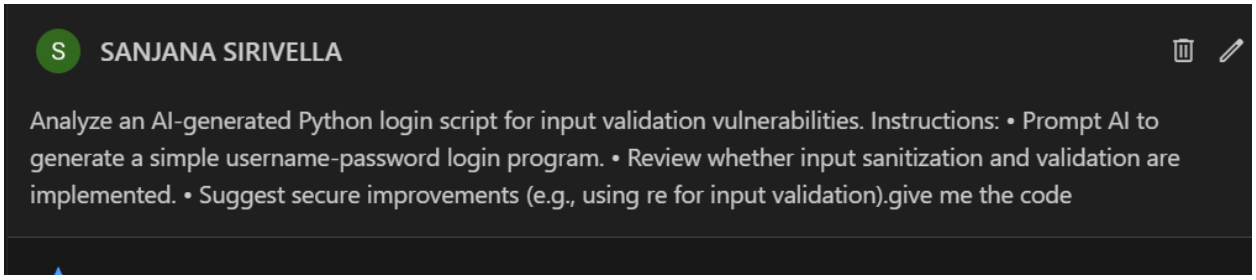
NAME:SIRIVELLA SANJANA

ROLL NO:2403A510D4

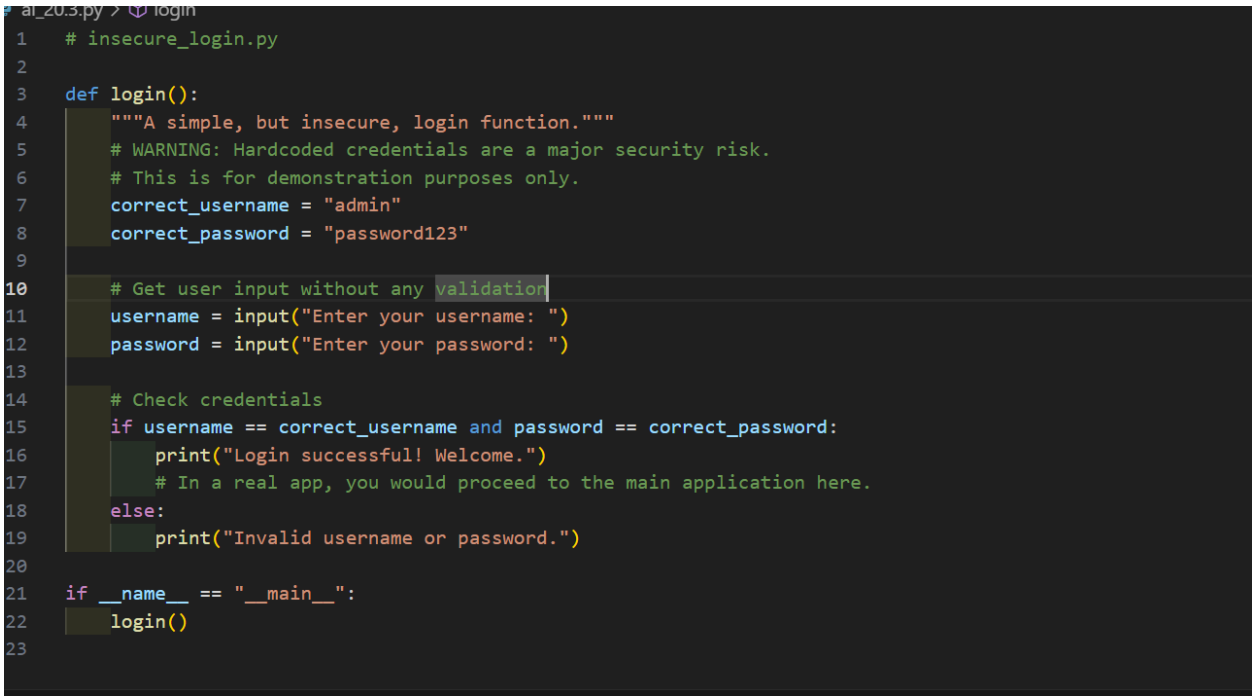
BATCH NO:05

TASK NO-1

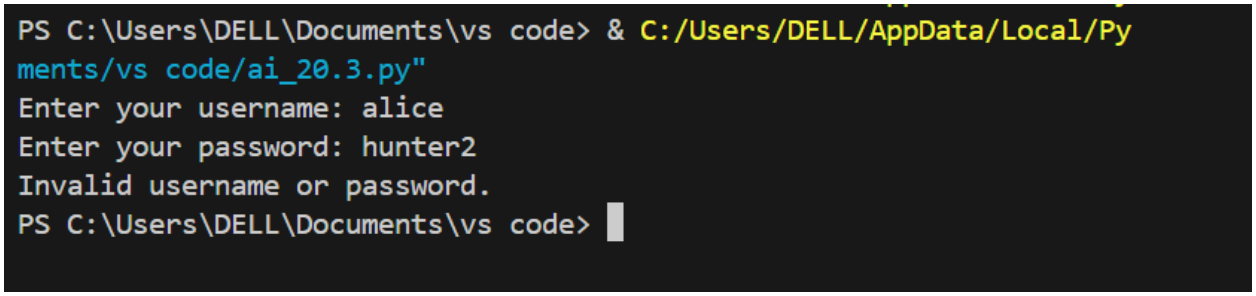
PROMPT:



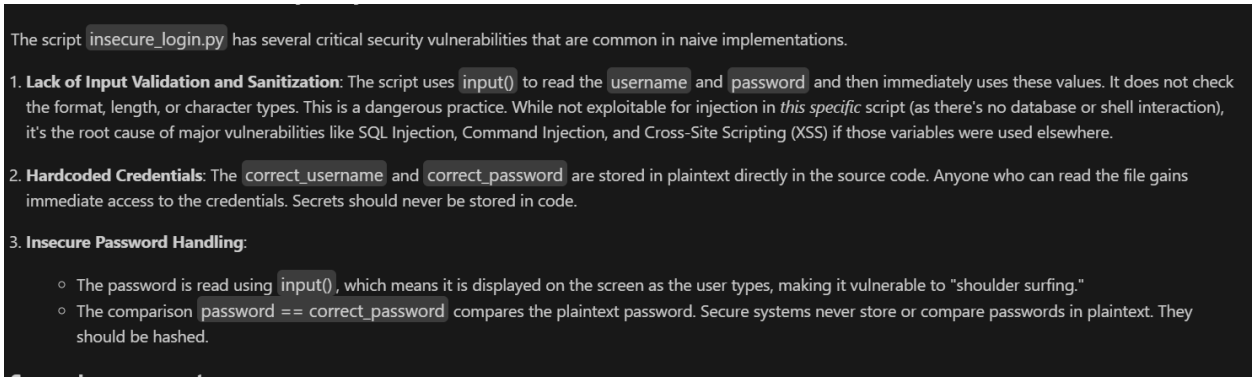
CODE:



OUTPUT:



OBSERVATIONS:

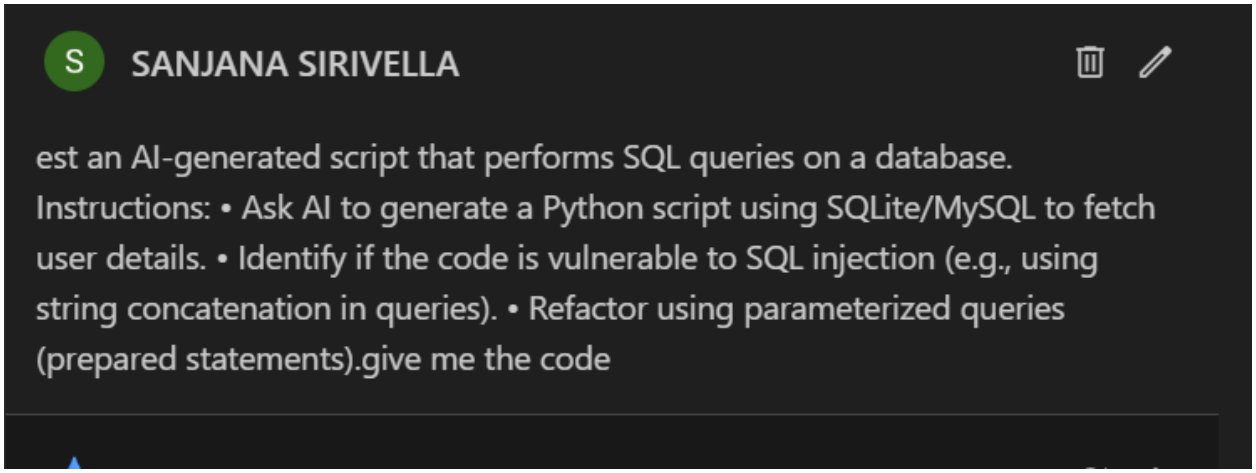


The script `insecure_login.py` has several critical security vulnerabilities that are common in naive implementations.

1. **Lack of Input Validation and Sanitization**: The script uses `input()` to read the `username` and `password` and then immediately uses these values. It does not check the format, length, or character types. This is a dangerous practice. While not exploitable for injection in *this specific* script (as there's no database or shell interaction), it's the root cause of major vulnerabilities like SQL Injection, Command Injection, and Cross-Site Scripting (XSS) if those variables were used elsewhere.
2. **Hardcoded Credentials**: The `correct_username` and `correct_password` are stored in plaintext directly in the source code. Anyone who can read the file gains immediate access to the credentials. Secrets should never be stored in code.
3. **Insecure Password Handling**:
   - The password is read using `input()`, which means it is displayed on the screen as the user types, making it vulnerable to "shoulder surfing."
   - The comparison `password == correct_password` compares the plaintext password. Secure systems never store or compare passwords in plaintext. They should be hashed.

TASK-02

PROMPT:



CODE:

```python
# vulnerable_user_lookup.py
import sqlite3

def setup_database(conn):
    """Creates a users table and inserts some sample data."""
    cursor = conn.cursor()
    cursor.execute("DROP TABLE IF EXISTS users")
    cursor.execute("""
        CREATE TABLE users (
            id INTEGER PRIMARY KEY,
            username TEXT NOT NULL,
            email TEXT NOT NULL,
            is_admin INTEGER
        )
    """)
    cursor.execute("INSERT INTO users VALUES (1, 'alice', 'alice@example.com', 0)")
    cursor.execute("INSERT INTO users VALUES (2, 'bob', 'bob@example.com', 0)")
    cursor.execute("INSERT INTO users VALUES (3, 'charlie_admin', 'charlie@example.com', 1)")
    conn.commit()

def get_user_details(conn, username):
    """
    Fetches user details from the database.
    WARNING: This function is VULNERABLE to SQL injection.
    """
    cursor = conn.cursor()
```

```python
def get_user_details(conn, username):
    # The vulnerability is here: using an f-string to build the query.
    query = f"SELECT id, username, email FROM users WHERE username = '{username}'"
    print(f"\nExecuting query: {query}")

    try:
        cursor.execute(query)
        user = cursor.fetchone()
        if user:
            print(f"User found: ID={user[0]}, Username={user[1]}, Email={user[2]}")
        else:
            print("User not found.")
    except sqlite3.Error as e:
        print(f"An error occurred: {e}")

if __name__ == "__main__":
    connection = sqlite3.connect(":memory:")
    setup_database(connection)

    # Get user input
    user_input = input("Enter the username to look up: ")

    # Fetch details
    get_user_details(connection, user_input)
```

```python

    connection.close()

```

OUTPUT:

```
Invalid username or password.
PS C:\Users\DELL\Documents\vs code> & C:/Users/DELL/AppData/Local/Python/pythoncore-3.14-64/python.exe "
vs code/ai_20.3.py"
Enter the username to look up: sanjana
vs code/ai_20.3.py"
Enter the username to look up: sanjana
Enter the username to look up: sanjana


Executing query: SELECT id, username, email FROM users WHERE username = 'sanjana'
User not found.
PS C:\Users\DELL\Documents\vs code>
```

OBSERVATIONS:

Vulnerability Breakdown:

1. **String Concatenation:** The script uses an f-string to directly insert the `username` variable into the SQL query. This means any text provided by the user becomes part of the SQL command executed by the database.
2. **Exploitation:** A malicious user can bypass the intended logic. For example, if a user enters the following input: `' OR '1'='1` The resulting query becomes: `SELECT id, username, email FROM users WHERE username = '' OR '1'='1'` Since `'1'='1'` is always true, the `WHERE` clause will be true for every row, and the query will return all users from the database, not just the one being searched for.

Secure Refactoring: Parameterized Queries

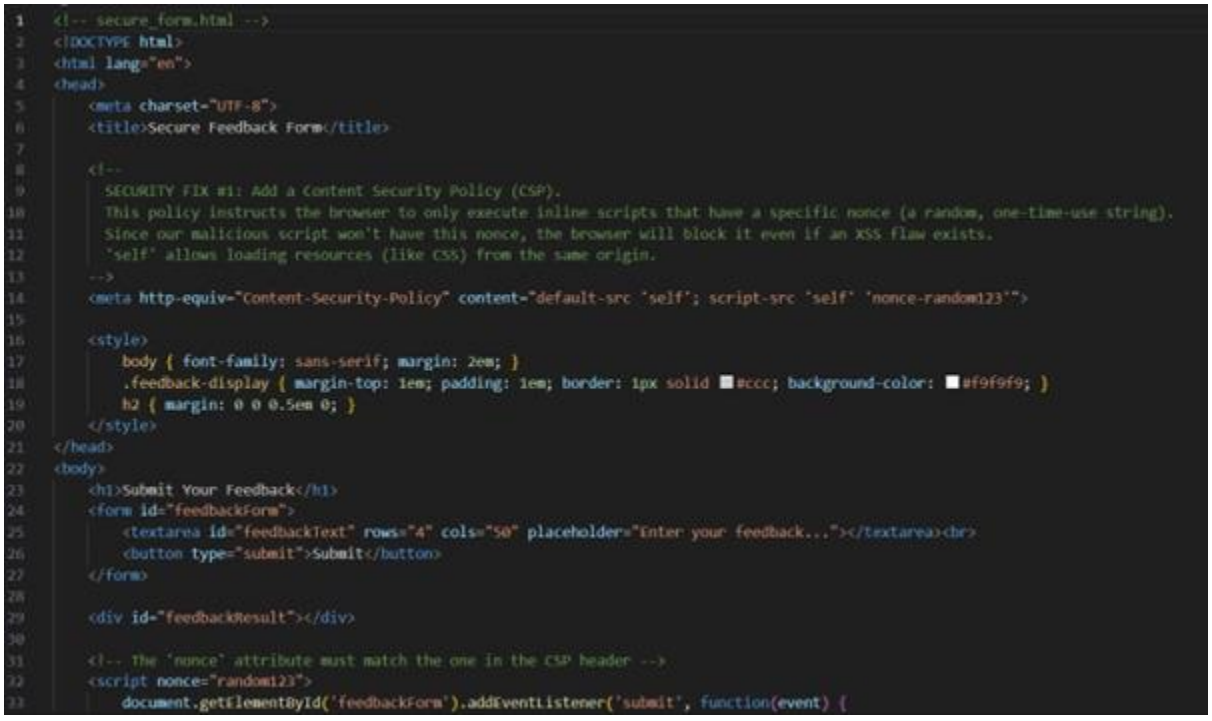
To fix this critical vulnerability, you must stop building queries with string formatting and instead use **parameterized queries** (also known as prepared statements). This practice separates the SQL command from the data, allowing the database driver to handle the data safely and prevent it from being interpreted as code.

TASK-03

PROMPT:

> "Generate an HTML page with a simple feedback form containing a single text area for comments and a submit button. When the user submits the form, use JavaScript to display the submitted feedback message directly on the page below the form without reloading the page."

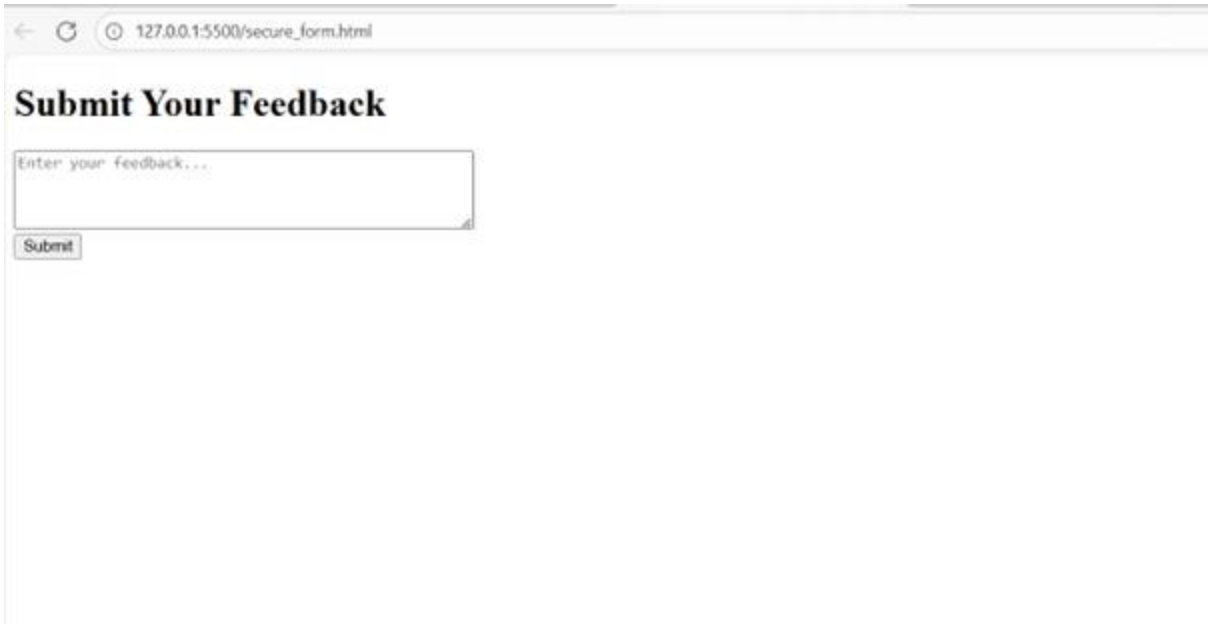
CODE:



```
<!-- secure_form.html -->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Secure Feedback Form</title>

    <!--
      SECURITY FIX #1: Add a Content Security Policy (CSP).
      This policy instructs the browser to only execute inline scripts that have a specific nonce (a random, one-time-use string).
      Since our malicious script won't have this nonce, the browser will block it even if an XSS flaw exists.
      'self' allows loading resources (like CSS) from the same origin.
    -->
    <meta http-equiv="Content-Security-Policy" content="default-src 'self'; script-src 'self' 'nonce-random123'">

    <style>
        body { font-family: sans-serif; margin: 2em; }
        .feedback-display { margin-top: 1em; padding: 1em; border: 1px solid #ccc; background-color: #f9f9f9; }
        h2 { margin: 0 0 0.5em 0; }
    </style>
</head>
<body>
    <h1>Submit Your Feedback</h1>
    <form id="feedbackForm">
        <textarea id="feedbackText" rows="4" cols="50" placeholder="Enter your feedback..."></textarea><br>
        <button type="submit">Submit</button>
    </form>

    <div id="feedbackResult"></div>

    <!-- The 'nonce' attribute must match the one in the CSP header -->
    <script nonce="random123">
        document.getElementById('feedbackForm').addEventListener('submit', function(event) {
```

```
        document.getElementById('feedbackForm').addEventListener('submit', function(event) {
            event.preventDefault();

            const feedback = document.getElementById('feedbackText').value;
            const resultDiv = document.getElementById('feedbackResult');

            // SECURITY FIX #2: Use .textContent instead of .innerHTML.
            // .textContent treats the input as plain text, automatically escaping any HTML characters.
            // This prevents the browser from interpreting it as code.
            resultDiv.innerHTML = '<h2>Your Feedback:</h2>'; // It's safe to set static HTML this way.
            const feedbackParagraph = document.createElement('p');
            feedbackParagraph.textContent = feedback; // The user input is safely assigned here.
            resultDiv.appendChild(feedbackParagraph);
        });
    </script>
</body>
</html>
```

OUTPUT:



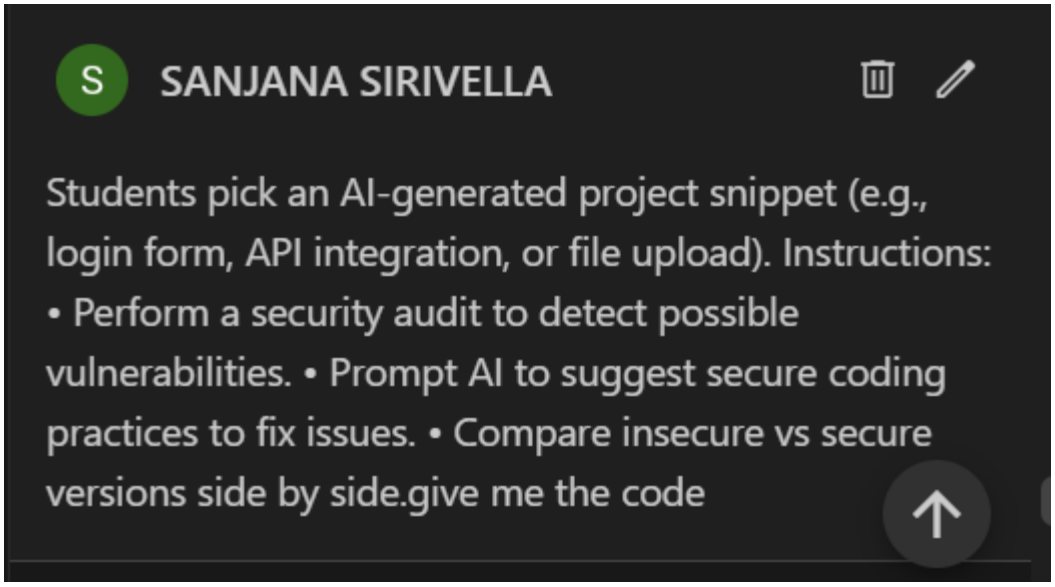
OBSERVATIONS:

The script is vulnerable to a **Stored Cross-Site Scripting (XSS)** attack.

- **Root Cause:** The vulnerability is in this line of JavaScript: `resultDiv.innerHTML = '<h2>Your Feedback:</h2><p>' + feedback + '</p>';`. The `.innerHTML` property tells the browser to parse the string as HTML. When the `feedback` variable contains malicious HTML (like a `<script>` tag), the browser will execute it.
- **Attack Vector:** An attacker can submit a script as their feedback. Instead of text, they can input a payload like `<script>alert('XSS Attack!');</script>`.
- **Impact:** When the form is "submitted," the script is injected directly into the page's DOM and executed by the browser. This could be used to steal cookies, redirect the user to a malicious site, capture keystrokes, or perform actions on behalf of the user without their consent.

TASK-04

PROMPT:



CODE:

ai_20.3.py > ...

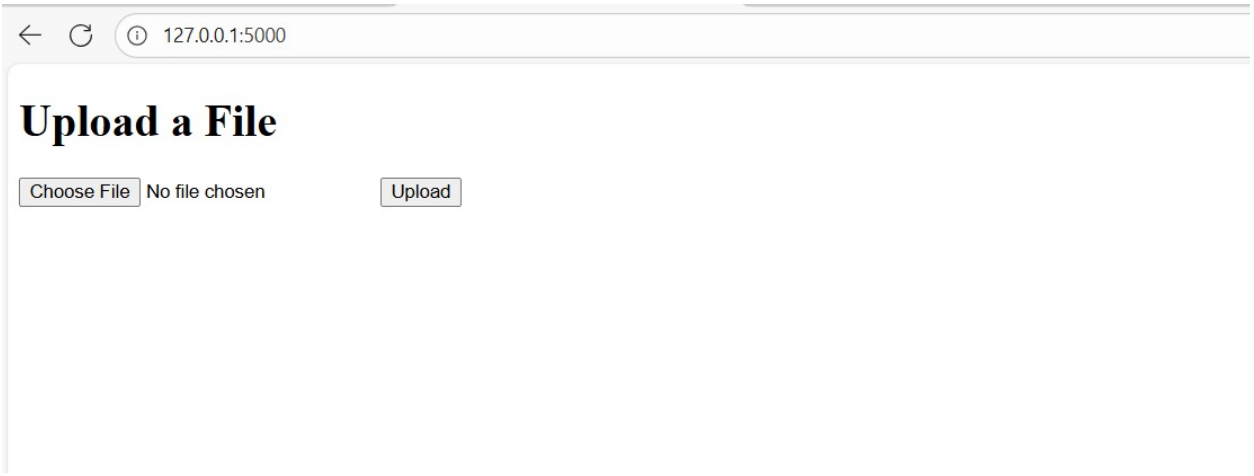
```python
# secure_upload.py
import os
import uuid
import logging
from pathlib import Path
from flask import Flask, request, abort, jsonify
from werkzeug.utils import secure_filename

# Configuration
BASE_DIR = Path(__file__).resolve().parent
# Keep uploads *outside* any static / served folder. This demo uses a subfolder but in prod
# you should use a dedicated storage (S3, object store) or a directory not served by the webse
UPLOAD_FOLDER = BASE_DIR / "uploads"
ALLOWED_EXTENSIONS = {".txt", ".pdf", ".png", ".jpg", ".jpeg", ".gif"}  # whitelist
MAX_CONTENT_LENGTH = 8 * 1024 * 1024  # 8 MB max upload size

app = Flask(__name__)
app.config["UPLOAD_FOLDER"] = str(UPLOAD_FOLDER)
app.config["MAX_CONTENT_LENGTH"] = MAX_CONTENT_LENGTH

# Ensure the upload folder exists with safe permissions
UPLOAD_FOLDER.mkdir(parents=True, exist_ok=True)
# set restrictive permissions: owner read/write only (may not be needed on windows)
try:
    os.chmod(UPLOAD_FOLDER, 0o700)
except Exception:
    # chmod may fail on some OSes; ignore but log
    pass

```

```python
# logging (don't print paths to stdout in prod)
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

def allowed_file_extension(filename: str) -> bool:
    _, ext = os.path.splitext(filename.lower())
    return ext in ALLOWED_EXTENSIONS

def safe_save_file(uploaded_file) -> str:
    """
    Saves file using a secure filename + random prefix (UUID) and validates path.
    Returns the saved file name (not full path).
    Raises werkzeug.exceptions to produce appropriate HTTP errors.
    """
    # 1) Use werkzeug's secure_filename to remove dangerous characters
    original_filename = uploaded_file.filename
    safe_name = secure_filename(original_filename)
    if not safe_name:
        abort(400, "Invalid filename.")

    # 2) Enforce extension whitelist (defense-in-depth)
    if not allowed_file_extension(safe_name):
        abort(400, "File type not allowed.")

    # 3) Generate random filename to avoid collisions and to avoid exposing original filename
    unique_name = f"{uuid.uuid4().hex}_{safe_name}"

    # 4) Resolve final path and ensure it's inside UPLOAD_FOLDER (prevent path traversal)
    final_path = Path(app.config["UPLOAD_FOLDER"]) / unique_name
    try:
        final_resolved = final_path.resolve(strict=False)
        upload_dir_resolved = Path(app.config["UPLOAD_FOLDER"]).resolve(strict=True)
    except Exception:
        abort(500, "Server file system error.")

    if upload_dir_resolved not in final_resolved.parents and final_resolved != upload_dir_reso
        # final path is not inside upload dir -> possible path traversal
        abort(400, "Invalid file path.")

    # 5) Save file
    uploaded_file.save(str(final_resolved))

    # 6) Set safe file permissions (owner read/write)
    try:
        os.chmod(final_resolved, 0o600)
    except Exception:
        # ignore failures on systems that don't support chmod in the same way
        pass

    # Return just the filename we used (not full path) so we don't leak server paths
    return unique_name

@app.route("/", methods=["GET"])
def index():
    return (
        """<!doctype html>
```

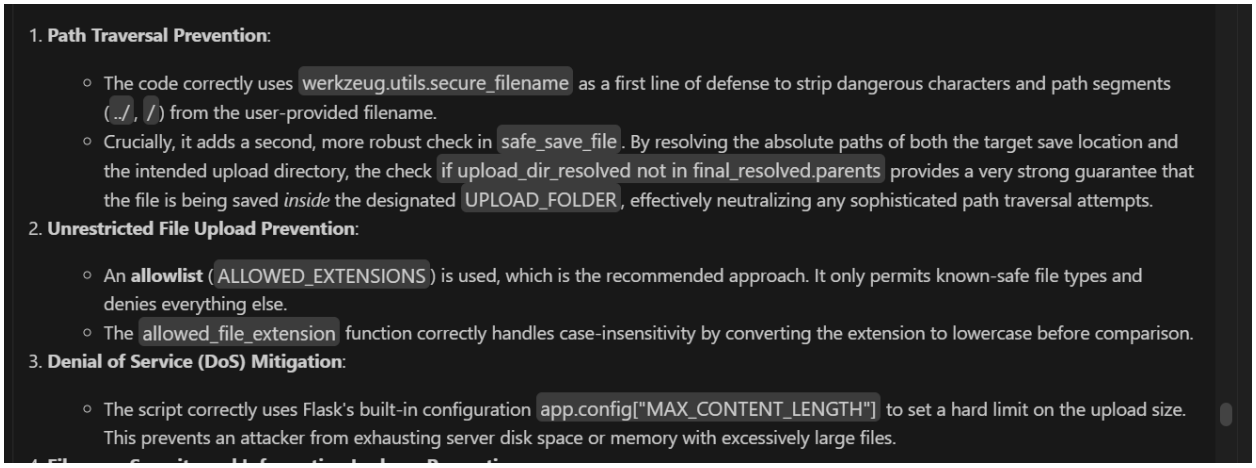
```
        """<!doctype html>
    <title>Secure File Upload</title>
    <h1>Upload a File</h1>
    <form method="post" action="/upload" enctype="multipart/form-data">
      <input type="file" name="file">
      <input type="submit" value="Upload">
    </form>
    """
    )

@app.route("/upload", methods=["POST"])
def upload_file():
    # 1) Check that the request actually contains a file part
    if "file" not in request.files:
        return jsonify({"error": "No file part in the request."}), 400

    file = request.files["file"]

    # 2) Basic checks
    if file.filename == "":
        return jsonify({"error": "No file selected."}), 400

    # 3) Optional: Basic MIME check (not fully reliable; combine with extension whitelist)
    mimetype = file.mimetype or ""
    if not any(mimetype.startswith(mt) for mt in ("image/", "application/", "text/")):
        # Allow only common safe top-level types — keep this conservative
        return jsonify({"error": "Unsupported media type."}), 400
```

```
def upload_file():

    # 4) Save file securely
    try:
        saved_name = safe_save_file(file)
    except Exception as exc:
        # abort will raise appropriate HTTP responses; catch generic to log
        logger.exception("Failed to save upload")
        # If it's an HTTPException raised by abort(), re-raise so Flask handles it properly
        from werkzeug.exceptions import HTTPException
        if isinstance(exc, HTTPException):
            raise
        abort(500, "Failed to save file.")

    # 5) Log safely (do not log full filesystem path or original filename in production)
    logger.info("Saved uploaded file as %s", saved_name)

    # 6) Respond without leaking server filesystem paths
    return jsonify({"message": "File uploaded successfully.", "filename": saved_name}), 201

if __name__ == "__main__":
    # For development only. In production:
    # - run behind a proper WSGI server (gunicorn/uvicorn) and a reverse proxy
    # - do NOT enable debug=True
    app.run(host="127.0.0.1", port=5000, debug=False)
```

OUTPUT:



OBSERVATIONS:



1. **Path Traversal Prevention**:
   - The code correctly uses `werkzeug.utils.secure_filename` as a first line of defense to strip dangerous characters and path segments (`../`, `/`) from the user-provided filename.
   - Crucially, it adds a second, more robust check in `safe_save_file`. By resolving the absolute paths of both the target save location and the intended upload directory, the check `if upload_dir_resolved not in final_resolved.parents` provides a very strong guarantee that the file is being saved *inside* the designated `UPLOAD_FOLDER`, effectively neutralizing any sophisticated path traversal attempts.
2. **Unrestricted File Upload Prevention**:
   - An **allowlist** (`ALLOWED_EXTENSIONS`) is used, which is the recommended approach. It only permits known-safe file types and denies everything else.
   - The `allowed_file_extension` function correctly handles case-insensitivity by converting the extension to lowercase before comparison.
3. **Denial of Service (DoS) Mitigation**:
   - The script correctly uses Flask's built-in configuration `app.config["MAX_CONTENT_LENGTH"]` to set a hard limit on the upload size. This prevents an attacker from exhausting server disk space or memory with excessively large files.